

INDY 1 - Study Buddy

Final Report

Professor Perry

CS 4850 - Section 03 - Fall 2025

Dec 8, 2025

Ojerhomu Erakpotobor, Jasper Min, Kara Welch

Lines of Code Written: ~2,100 - 2,250 lines of code (backend + frontend)

Number of Project Components/Tools (List):

Total Man Hours:

Project is ~95% Complete

Git: <https://github.com/ojerhomu/StudyBuddy/>

website: <https://ojerhomu.github.io/StudyBuddy/website/>

Table of Contents

1. Introduction + Abstract	3
2. Challenges & Assumptions	4
2.1. Key Assumptions	4
2.2. Project Challenges	4
3. Requirements	5
3.1. Functional Requirements	5
3.2. Non-Functional Requirements	7
4. Analysis/Design	8
4.1. System Architecture Analysis (Client-Server Model)	8
4.2. Data Management and Backend Design	9
4.3. AI and Scheduling Component Design	10
5. Development	11
5.1. Project Setup and Directory Structure	11
5.2. Database Connection and ORM Implementation	11
5.3. Data Models and FastAPI Integration	11
5.4. AI Integration and Development Dependencies	11
6. Test Plan	13
6.1. Scope and Objectives of Testing	13
6.2. Specialized Testing and Risk Mitigation	13
6.3. Data Persistence testing (This is top priority)	13
6.4. AI Homework Help +Quiz Chatbot Testing	14
6.5. Calendar + Event Management Testing	14
6.6. Class Schedule Management Testing	14
7. Test Report	15
7.1. Test Report Summary and Execution	15
7.2. Integration and Security Validation	15
7.3. AI Integration and Performance Verification	15
8. Version Control	16
8.1. Benefits of Version Control	16
8.2. Repository Structure	16
8.3. Branching Strategy	17
9. Conclusion	18
10. Appendix	19
10.1. Appendix A- Mobile App Training Guide	21
10.2. Appendix B- Screenshots	22
10.3. Appendix C- Gantt Chart from our Project Plan	25
10.4. Appendix D- Test Environment Map	25

1. Introduction + Abstract

The Study Buddy application is a mobile app designed for a target audience of undergraduate and graduate college students to help them effectively manage large workloads by scheduling study sessions and organizing class materials.

The application has a client-server architecture, using Kotlin Android frontend for the user interface and a Python backend with FastAPI framework to manage server logic. Data persistence is handled by a PostgreSQL database, with communication completed by the SQLAlchemy Object Relational Mapper. The application's key functionalities include its secure user authentication, which utilizes bcrypt password hashing and JSON Web Tokens, and comprehensive task management that stores curriculum elements and user preferences.

The project is centered on integrating artificial intelligence via the Gemini 2.5 API, which is tasked with analyzing user-defined deadlines and available time blocks to automatically generate and recommend optimized study sessions. Overall development prioritized a secure, scalable, and intelligent platform for enhanced student productivity.

2. Challenges & Assumptions

2.1. Key Assumptions

The project relies on the fundamental assumption of certain user behavior and technical environments. A primary assumption is that the target user, likely an undergraduate or graduate college student, will be competent in using a mobile application for academic management and will consistently provide accurate data regarding their class schedule, task deadlines, and available study blocks.

The system's core scheduling mechanism, which relies on the Gemini 2.5 AI, assumes the model will be capable of reliably processing and interpreting this data. Furthermore, the selection of a client-server model assumes that users will have reliable internet connectivity to ensure seamless communication between the Kotlin mobile front end, the FastAPI Python backend, and the PostgreSQL database for real-time task management, authentication, and AI service calls.

Finally, from a development standpoint, it is assumed that the external dependencies, such as the Gemini 2.5 API Key generated through Google Firebase, will maintain continuous support and stable service throughout the application's lifecycle.

2.2. Project Challenges

The project faces challenges primarily related to system integration and the complexity of the AI. The most significant challenge lies in juggling attempting to ensure secure and seamless communication between the Kotlin client, the Python/FastAPI server, and the PostgreSQL database. Any failure in the API endpoints or the JSON Web Token authentication process could critically compromise the user experience and system integrity.

Another substantial hurdle is the development and reliable performance of the AI scheduling algorithm. Moving beyond the initial proof-of-concept, which only generated a simple custom greeting, to a functional system requires significant effort in prompting the Gemini 2.5 API to handle the nuanced, real-world constraints of a student's schedule, including conflicting deadlines, varying course priorities, and unexpected time constraints. This demands robust input validation and meticulous error handling to prevent the AI from generating impractical or detrimental study plans.

3. Requirements

3.1. Functional Requirements

At the core of the system is secure user authentication, which requires the backend to support encrypted password storage using bcrypt hashing and the issuance of JSON Web Tokens (JWTs) to maintain a secure and active user session. Users must be able to register an account, log in, and maintain an authenticated state while using the mobile application. The backend must validate all protected routes, such as those handling tasks, calendar events, class schedules, and personal study preferences, all using the token provided by the client.

For academic organization, the application allows users to create, delete, and modify academic tasks, such as homework and exams, and store user preferences such as course priorities and available study times in the database. The most critical functional requirement involves the Gemini 2.5 AI component's intelligent scheduling system, which must successfully analyze the meshing of task deadlines and stored user preferences to automatically generate and recommend optimized study sessions to the user. All these operations must be handled seamlessly through the backend's API routes.

The system must allow the long-term storage of user interactions through the chat session feature. Each AI-based conversation is recorded in the backend's database to support progress tracking, reflection, or future scheduling enhancements.

The frontend has its own set of functional requirements. The mobile application has to provide a smooth, responsive user interface using Jetpack Compose, presenting screens for onboarding, study sessions, course setup, and calendar visualization. ViewModels must manage UI state across configuration changes and ensure that user preferences such as study duration, break times, and timer progress are saved and restored. Retrofit, combined with a TokenInterceptor, must send every backend request with the up-to-date JWT token, guaranteeing that communication with the backend is authenticated.

Those are some thought-out examples. We have included all functional requirements below:

User Account + Auth

- **R-1:** The system allows users to create an account using an email and a password
- **R-2:** The system hashes all passwords using bcrypt before storing them
- **R-3:** The system authenticates users and issues a JSON Web Token (JWT) upon successful login
- **R-4:** The system validates a user's JWT before granting access to protected resources
- **R-5:** The Android client automatically attaches the JWT to all backend requests via a TokenInterceptor

User Profile + Preferences

- **R-6:** The system allows users to update profile information, including first name, last name, and education level
- **R-7:** The system allows users to store and modify Pomodoro preferences (study duration, break durations)
- **R-8:** The system loads user preferences on app launch and persists them across sessions

Tasks + Assignments

- **R-9:** Users will be able to create academic tasks, including title, description, and due date
- **R-10:** Users should be able to view all tasks associated with their account.
- **R-11:** Users should be able to delete tasks
- **R-12:** Each task is stored in the database and linked to the current authenticated user

Calendar Management

- **R-13:** Users should be able to create calendar events with a start time, end time, title, description, and event type
- **R-14:** The system retrieves all calendar events belonging to the authenticated user
- **R-15:** Users should be able to delete calendar events from the backend
- **R-16:** All events are persistently stored and associated with the correct user ID

Class Schedule Management

- **R-17:** Users will be able to create and store up to seven courses
- **R-18:** The system allows each course to have a color or subject category
- **R-19:** Users should be able to create class schedules containing day-of-week, start time, and end time values
- **R-20:** The system validates that asynchronous classes do not contain time fields
- **R-21:** The system allows schedules to be updated and prevents duplicate entries

AI Chatbot Homework help and Quizing

- **R-22:** The system provides an AI chatbot (El the elephant) capable of answering homework-related questions interactively
- **R-23:** The chatbot guides students toward answers using hints, probing questions, and step-by-step reasoning
- **R-24:** The chatbot has conversational context using Gemini's multi-turn chat API
- **R-25:** The system stores AI chat sessions persistently in the backend

- **R-26:** Users should be able to resume previous chatbot sessions by loading prior messages
- **R-27:** The AI allows users to request a quiz by specifying their name, subject, and topics
- **R-28:** The AI generates a ten-question quiz based on the selected topic list.
- **R-29:** The AI collects answers, one question at a time
- **R-30:** The AI evaluates all answers after the quiz flow is complete
- **R-31:** The AI offers users the option to review incorrect questions with explanations

3.2. Non-Functional Requirements

For non-functional requirements, security is a main concern. This is the project's most critical non-functional concern due to the sensitivity of user data and the reliance on authentication tokens. All passwords must be stored using strong, non-reversible bcrypt hashing, and the backend must enforce strict JWT validation for every protected route. In addition, communication between the Android client and the FastAPI backend must occur through HTTPS in production environments to prevent token interception or credential leakage. Input validation at both the backend and frontend layers protects against malformed data, injection attacks, and inconsistent scheduling input formats.

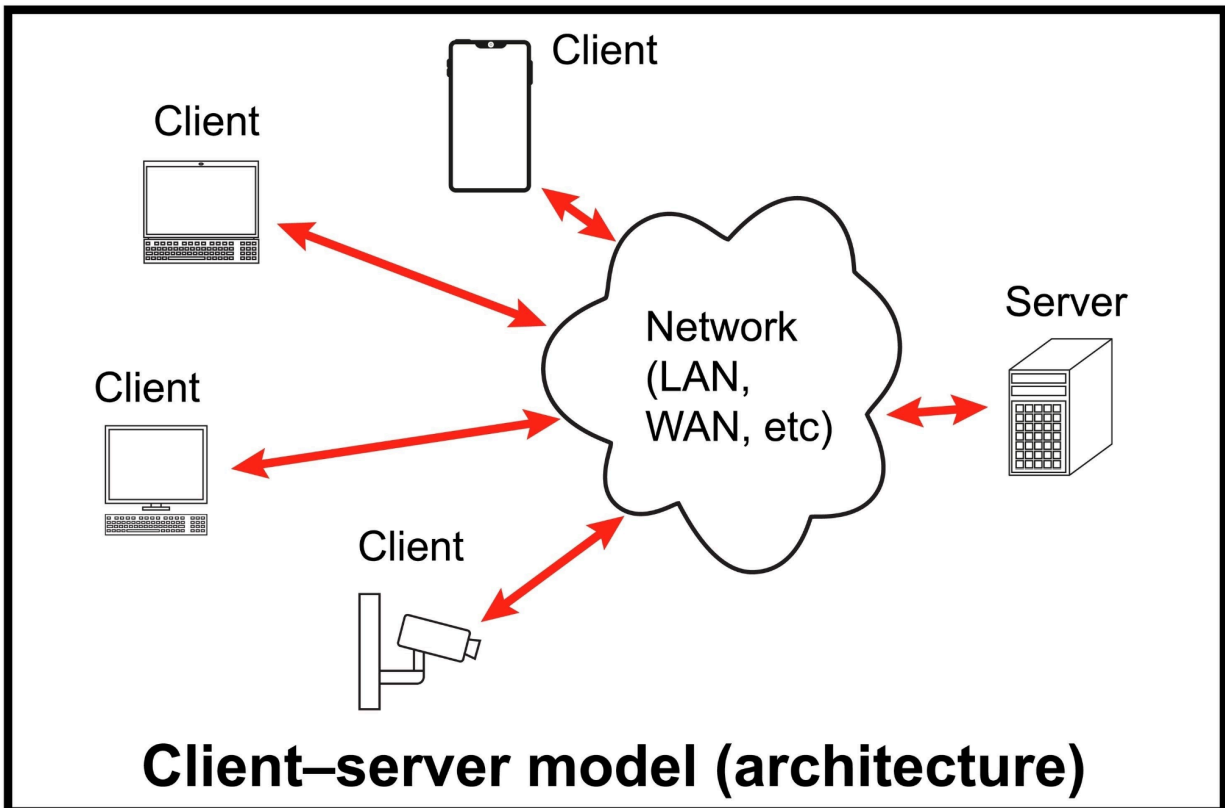
The application must be available whenever students need it, The FastAPI backend has to respond efficiently to user actions such as retrieving tasks, creating calendar events, updating class schedules, and loading user profiles. SQLAlchemy queries must execute with little latency to provide a positive experience for students who may interact with the application multiple times throughout the day. On the frontend, Jetpack Compose must maintain smooth animations, immediate UI feedback, and responsive screen transitions even when handling real-time timer updates or AI-generated content. The TimerViewModel must update countdown values in an efficient way using StateFlow to prevent unnecessary recompositions, ensuring stable performance across various device types. The database must be persistent to ensure no loss of critical user data or academic tasks. On the frontend, the app must gracefully handle loss of internet connectivity by presenting clear error messages and preventing corrupted or partial API requests.

Finally, we have to consider scalability for potential future development. The architecture must support an expanding user base without sacrificing any performance. The backend's modular router design and PostgreSQL relational schema allow additional features, such as a potential AI scheduling feature where the AI suggests when you work on certain tasks. We could consider group study coordination as a new feature as well. With these features in mind, we want them to be integrated without major architectural changes. Horizontal scaling is achievable through FastAPI's ASGI compatibility with modern servers like Uvicorn, while PostgreSQL can be migrated to cloud-hosted environments if demand ever increases.

4. Analysis/Design

4.1. System Architecture Analysis (Client-Server Model)

The design is rooted in a three-tiered client-server model selected for its scalability, maintainability, and ability to separate presentation logic from business logic and data management.



The Kotlin Android client, on tier 1, was chosen for mobile development due to its modern capabilities and high compatibility with the Gemini 2.5 API. This client handles all UI/UX elements, transmitting user inputs such as tasks, preferences, and credentials to the backend.

The Python/FastAPI backend, on tier 2, serves as the application server, acting as the critical middleman. Its key responsibilities include routing client requests, executing business logic such as password hashing via bcrypt and JSON Web Token generation, and hosting the AI scheduling algorithm. This separation ensures that security sensitive operations are abstracted away from the mobile device, enhancing both performance and security.

4.2. Data Management and Backend Design

We've built the backend to be modular, secure, and scalable for future development. We wanted a clean separation between authentication, routing, data modeling, and storage. PostgreSQL serves as the system's persistent storage engine, selected for its reliability, robustness, and ability to scale as the user base grows. SQLAlchemy ORM was integrated to map Python classes to database tables, to make sure that models such as User, Task, Event, etc. can be manipulated through object-oriented logic rather than raw SQL queries (so the group would spend less time learning a new language). This abstraction not only increases code readability but also reduces the likelihood of database inconsistencies.

The database connection is managed through a dedicated module (`database.py`), which initializes the SQLAlchemy engine using a secure environment variable (`DATABASE_URL`). A session generator function (`get_db`) ensures that each API request receives its own database session, which is opened and closed. This follows safe transaction handling and prevents resource leakage across concurrent requests. The modular router system within FastAPI organizes all API endpoints logically by domain tasks, calendar events, users, schedules, and chats, which makes the backend easier to maintain and expand.

With data management, we wanted to include validation layers implemented through Pydantic schemas. These schemas enforce correct data formats before any database operation occurs, all to make sure we prevent issues like invalid dates, bad schedule entries, or incorrect course identifiers. The design has complex validation scenarios in mind, such as ensuring that asynchronous classes do not include start or end times, or automatically normalizing day-of-week inputs to maintain consistency across the database.

In terms of security, the `auth.py` module handles bcrypt password hashing, token creation, and current-user authentication logic. When a user logs in, their credentials are validated, and a JWT token is generated using a secure signing key and expiration time. Every protected endpoint uses dependency injection to validate this token before performing operations. This ensures that users can only interact with their own tasks, events, and class schedules, preventing unauthorized access to sensitive data.

This modular and layered design means the backend can easily integrate new features in later phases of development, such as AI-driven schedule generation or cross-user collaboration tools. The combination of FastAPI, SQLAlchemy, PostgreSQL, and JWT authentication provides a secure and maintainable foundation capable of supporting evolving academic needs for students.

4.3. AI and Scheduling Component Design

The most complex aspect of the design is the integration of the Gemini 2.5 API to provide intelligent scheduling. The decision to use a large language model like Gemini was based on its demonstrated capability to handle complex, nuanced constraints, which was a requirement for turning a collection of deadlines and availability blocks into an optimized study schedule.

On the mobile frontend, AI interactions occur within dedicated ViewModels that handle network requests, generate properly formatted prompts, and manage UI state. For example, the Homework Help chatbot guides students toward answers by asking questions, offering hints, and breaking down complex problems into manageable steps, all framed through a system prompt that defines EI as a patient and encouraging study companion. The Quiz Me feature follows a more procedural flow, where the AI generates a ten-question quiz based on the user's selected subject and topics, collects answers one at a time, evaluates performance, and optionally reviews missed questions with explanations. Both systems leverage persistent multi-turn chat sessions, allowing the app to save conversations through backend APIs and reload them with full context when resumed. This creates a personalized study experience where the AI has context about the user beforehand.

The backend validates all data used for scheduling through existing Pydantic schemas to ensure that the AI receives inputs with no errors. For example, class schedules must first be normalized into consistent day-of-week strings and validated time blocks. Deadlines retrieved from user tasks will be passed through a sorting and grouping process so the AI receives a clear sense of urgency and workload distribution. This ensures that the resulting study schedule is logical and aligned with good academic practices.

For future scheduling functionality, the design requires the app to compile structured academic data such as deadlines, class schedules, task priorities, estimated workload, and available time blocks all into a sanitized JSON-like prompt format. This data will be transmitted to the backend, which is responsible for formatting and routing the AI request to Gemini.

5. Development

5.1. Project Setup and Directory Structure

The setup of our backend environment required Python and key libraries such as FastAPI, SQLAlchemy, bcrypt for password hashing, and the Python Google GenAI library for AI integration. The backend code was segmented into separate files - main.py for initialization and routes, auth.py for authentication and JWT handling, models.py for database tables, schemas.py for data validation, and database.py for connection logic.

The client frontend required Kotlin and was developed within Android Studio. Setting up the project involved linking the main repository, managed via GitHub, to Firebase to generate the necessary Gemini 2.5 API Key, and integrating Google SDKs into the mobile project dependencies. This structured approach, documented through the specific file listing, ensured clarity and collaboration among the team members, with GitHub serving as the central hub for all code changes and version management.

5.2. Database Connection and ORM Implementation

The connection mechanism between the FastAPI backend and the PostgreSQL database is a critical part of the application's architecture. This process is managed by SQLAlchemy, which acts as the Object Relational Mapper. The connection is established in a dedicated module (database.py), which uses SQLAlchemy's create_engine function.

This function is supplied with a database connection string containing the dialect, driver, username, password, host, and database name (postgresql+psycopg2://user:password@host/dbname). Once the connection Engine is created, a SessionLocal class is generated using sessionmaker.

5.3. Data Models and FastAPI Integration

The ORM model design was important for our project's data integrity, as tables for User, Task, and Event were defined by Python classes inheriting from the SQLAlchemy Base. This model allowed us to manipulate the database records using standard object-oriented programming rather than writing raw SQL - For example, a User class would define attributes like username, email, and a column for the bcrypt-hashed password.

The FastAPI framework could then utilize Dependency Injection by using the get_db function within its API endpoints. This ensures that every incoming API request that needs database access, such as a user registering or retrieving their tasks, automatically receives a live database Session, executes its query, and reliably closes the session upon completion, thereby efficiently managing database resources and transactions.

5.4. AI Integration and Development Dependencies

A key development consideration was the integration of the AI component using the Gemini 2.5 API. This required adding the appropriate Google SDKs and the Google Genai Python library to the project's dependencies.

The development effort first established a proof of concept to confirm the operational integrity of the API key and connection before tackling the complex core functionality. This validation of the AI integration, alongside the structured use of GitHub for version control, were fundamental to the development process, ensuring that the team could track progress and debug issues effectively across the multiple interconnected technologies being used to create the application.

6. Test Plan

6.1. Scope and Objectives of Testing

The Test Plan for the Study Buddy application serves as a comprehensive quality assurance framework, defining the strategy, scope, and objectives for validating the system's integrity, security, and functionality. It outlines a systematic approach to quality assurance, focusing on key areas that directly impact the user experience and system reliability.

The primary testing categories include Unit Testing, Integration Testing, and Functional Testing. Unit Testing is designed to validate individual code components, such as specific functions within the backend and the reliability of the bcrypt password hashing process. Integration Testing is particularly critical for this client-server architecture, verifying the secure and seamless communication flow between the mobile client, the Python/FastAPI server, and the database. Finally, Functional Testing ensures that all core features, including user login, JWT validation, and the transactional integrity of task and preference management, operate correctly according to the defined requirements.

6.2. Specialized Testing and Risk Mitigation

Beyond the core categories, the Test Plan also employs several testing protocols to mitigate critical risks. Security Testing is performed to validate the robustness of the authentication system, specifically confirming the integrity of JWT tokens and the proper hashing of user passwords, while also defending against malicious database inputs.

To ensure the application could scale to accommodate a hypothetical large student body, Stress & Volume Testing and Connectivity Testing are included in the plan, verifying that the system maintains performance and stable network communication under heavy load conditions.

Furthermore, a strategy for Regression Testing has been implemented to be performed before any new features are integrated, particularly for the future AI scheduling assistant. This ensures that development efforts, such as enhancing the Gemini 2.5 API integration, do not inadvertently introduce defects or compromise the operational stability of existing core functionalities like user login or database persistence.

6.3. Data Persistence testing (This is top priority)

Data persistence test focuses on confirming that user information such as chat sessions, schedules, tasks, etc, is stored consistently in PostgreSQL with proper relational integrity. Testers can confirm that each database record includes correct foreign keys and that unauthorized access attempts are blocked by backend permission checks. Test scenarios deliberately introduce invalid or incomplete payloads to verify that the system prevents bad data from entering the system. These tests ensure that persistent storage remains reliable even under stress, incorrect user input, or unexpected failure conditions.

6.4. AI Homework Help + Quiz Chatbot Testing

Testing the homework help chatbot would focus on validating the step-guided assistance provided by EI, the AI. Testers could verify that starting a new homework conversation generates an appropriate AI response, and that the chatbot adheres to system instructions by offering hints, guiding questions, and step-by-step support rather than simply providing full answers. Multi-turn conversation persistence is tested by sending successive messages and confirming that Gemini maintains context throughout the session.

The Quiz Me feature is tested to make sure it follows its multi-phase workflow. Testers would verify that providing a name, subject, and topics results in the generation of a ten-question quiz relevant to the chosen material. The system must then accept answers sequentially, asking for responses to one question at a time and preserving full context between questions. Once all answers are submitted, the chatbot should identify incorrect responses without revealing correct answers prematurely. If a user opts to review missed questions, the AI must supply correct answers along with explanations in a clear, instructional tone.

6.5. Calendar + Event Management Testing

Calendar and event testing focuses on ensuring that academic events such as study sessions, exams, and class times are accurately stored and retrieved. Testers could verify that events can be created with valid data, including title, start time, end time, and event type. Events with invalid configurations, such as an end time earlier than the start time, must be rejected by backend validation rules. The system must also ensure that users only see events linked to their own account, reinforcing data privacy guarantees.

6.6. Class Schedule Management Testing

Class schedule testing validates the integrity of the course and schedule creation, ensuring the backend correctly interprets and stores academic structures. Tests could verify that users can create up to seven courses (max), with the backend rejecting attempts to exceed this limit. Schedules must be created with valid days-of-week names and properly formatted start and end times, where invalid or ambiguous inputs trigger validation failures. Asynchronous courses are tested to ensure that the system accepts schedules without time fields and correctly distinguishes them from time-based classes. Additional testing verifies that duplicate schedule entries are not created and that updating

7. Test Report

7.1. Test Report Summary and Execution

Overall, testing demonstrated high operational integrity across the critical components of the system. Unit Testing verified the reliability of foundational security services, confirming that the bcrypt password hashing mechanism would correctly encrypt user credentials, and that individual FastAPI functions perform their intended logic. Functional Testing confirmed that all user-facing features, from the initial registration and login process to the creation and modification of academic tasks, operate correctly as well. Specifically, the secure issuance and validation of JWTs were validated, ensuring secure ongoing user sessions. The successful execution of these tests confirms that the application's core functionality is stable and ready for user interaction.

7.2. Integration and Security Validation

A core focus of the testing effort was Integration Testing, which successfully validated the seamless data flow across our three primary technological layers (the Kotlin Android client, the Python/FastAPI backend, and the PostgreSQL database). Test cases confirmed that the client could reliably send authentication requests and task updates to the backend, which in turn successfully processed the logic and interacted with the database using SQLAlchemy.

Furthermore, Security Testing verified the effectiveness of our implemented protective measures. The system demonstrated resistance to known vulnerabilities by confirming password hashing, and that the API endpoints correctly enforce JWT authentication, mitigating risks associated with unauthorized access and data tampering.

7.3. AI Integration and Performance Verification

Testing also included validation of the AI scheduling component, specifically focusing on the stability of the Gemini 2.5 API integration. The success of the initial proof of concept confirmed the operational integrity of the API key, network connectivity to the service, and the correct setup of the necessary SDKs and libraries. While the full, complex scheduling algorithm is a future development, this foundational testing confirms the technical viability of integrating the AI model.

Finally, initial Stress & Volume testing provided preliminary confidence that the FastAPI backend and PostgreSQL database could be able to maintain responsive performance and transactional consistency under a moderate user load, suggesting a scalable foundation for future growth. The test results indicate that the Study Buddy application is a stable, secure, and technologically sound platform ready for the next phase of development.

8. Version Control

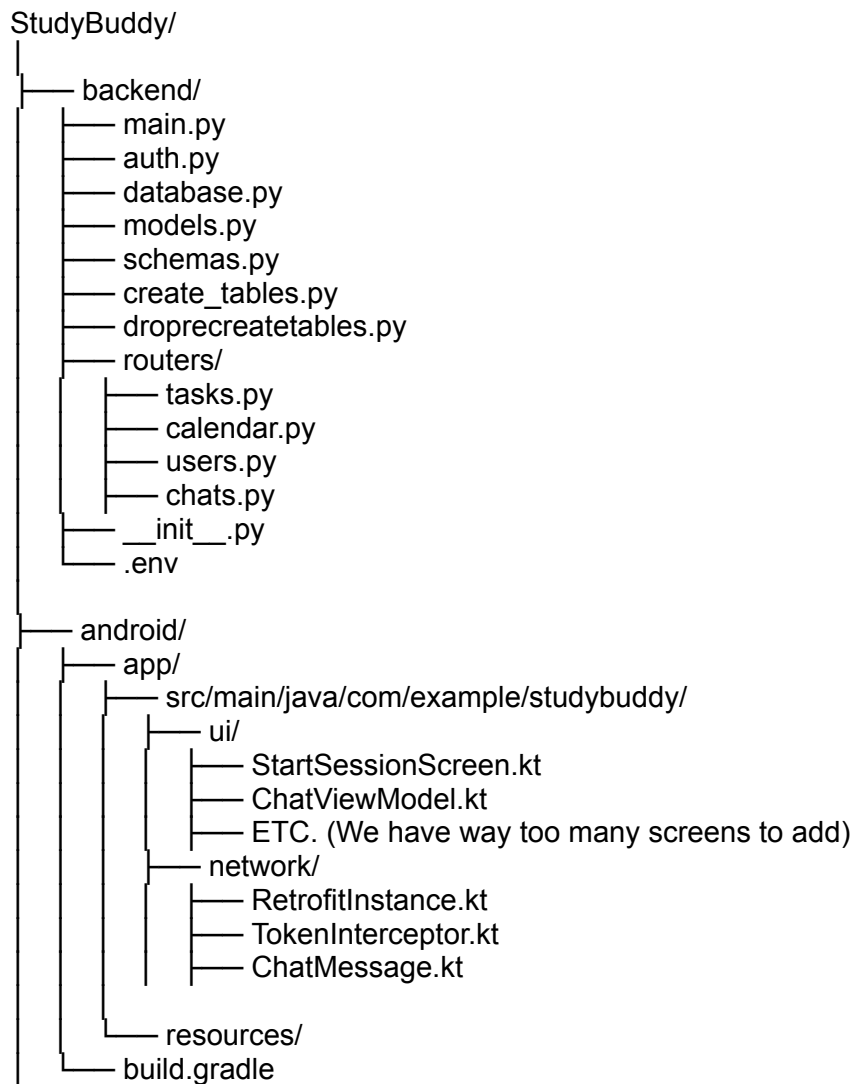
The Study Buddy project implemented GitHub as the central repository for version control, providing a collaborative environment essential for our multi-developer team. This system allowed us to manage separate branches for features (like the Kotlin client, FastAPI backend, and database models), ensuring that concurrent work could proceed without corrupting the main codebase.

Using GitHub facilitated tracking all source code changes, making it easy to revert to stable versions and enabling efficient peer review among our team members. Furthermore, the integration with Android Studio, which hosts the Kotlin front-end, streamlined the process of pulling the latest stable code, implementing necessary API calls, and pushing updated versions back to the repository. This structure was critical for maintaining continuity and stability as updated versions of the application were passed and merged among the developers.

8.1. Benefits of Version Control

Using GitHub allows our team to be in full control of the development, allowing multiple developers to work concurrently on different aspects of the system, such as backend API routing, authentication, Android UI screens, and AI integration, all without overwriting each other's work, unlike GoogleCollab. Version control offered safe experimentation through isolated branches, where new features could be developed, tested, and refined before being merged into the main repository. Version control also practices code safety and reliability, so we as developers can quickly revert changes without messing with the code for other developers.

8.2. Repository Structure



(pulled and modified from the Github repos documentation)

8.3. Branching Strategy

- The main branch represents the production-ready version of the application. Only fully tested, stable code is merged into this branch. It's essentially the reference point for final builds and documentation
- The development branch is the staging environment where features are integrated and tested together before being promoted to main. All team members merge their feature branches into development after review. This branch is used to verify compatibility between backend endpoints, Android UI, and AI integration.
- The feature branches are for implementing pieces of functionality including:
 - authentication
 - task-endpoints
 - ai-chatbot
 - ai-chatbot
 - etc.

9. Conclusion

The Study Buddy application successfully establishes a secure, scalable, and functional foundation for an intelligent student productivity tool. The project's development delivered a functioning client-server architecture, effectively linking a Kotlin Android front end to a Python/FastAPI backend, underpinned by a reliable PostgreSQL database.

Critical requirements, including bcrypt-secured user authentication, JWT session management, and robust task management, have been successfully implemented and validated through testing as well. The early integration of the Gemini 2.5 API confirms the technical viability of the central goal, leveraging AI to automatically generate optimized study schedules based on user-defined constraints.

The successful completion of the initial development and testing phases, which include rigorous Integration Testing and dedicated Security Testing provide a high degree of confidence in the application's stability and reliability. The architecture is deliberately designed to be modular and scalable, ensuring the system can evolve to support a growing user base and accommodate future feature expansions.

The next phase of development will focus entirely on enhancing the AI scheduling algorithm to its full potential, moving to deliver personalized study recommendations, thereby fulfilling the Study Buddy's ultimate promise to help college students effectively manage complex academic workloads.

10. Appendix

10.1. Appendix A- Mobile App Training Guide



Welcome to the Study Buddy! I'm El, your personal study partner!



I'm here to help you stay organized, learn more effectively, and build strong study habits throughout the semester. This guide will walk you through how to use the mobile app so you always know where to find your class schedule, due dates, and other essentials you need for organizing your classes. Let's get started!

1. Getting Started: Logging In and Setting Up

Once you open the app, you'll be asked to **log in or create an account**. I'll use your name and profile information to personalize your study experience. Creating an account will walk you through setting up your preferences, which I'll remember!

After signing in, I'll guide you through setting your **Pomodoro study preferences**, which is basically how long you like to study, how long your breaks are, and how often you need a long break. These settings help me time your study sessions.

2. Home Screen Navigation

From the main menu, you can choose from several tools I help you with:

- Study Session Timer
- Homework Help Chat
- Quiz Me Study Tool
- Task List + Calendar
- Class Schedule Setup

Just tap the feature you need, and I'll walk you through the rest.

3. Study Session Timer

When you start a study session, I'll display a timer matched to your preferences. During your session:

- I'll keep track of your study and break cycles
- I'll give you gentle encouragement along the way
- You can pause or stop at any time

After each session, I'll let you know when it's time for a break or when you've earned a long rest.

4. Homework Help Chat

Whenever you need help with a problem, just open the **Chat** section and send me a message.

How I help you:

- I won't give answers directly; I'll guide you step by step
- I'll ask questions that help you think through the problem
- I'll break big ideas into smaller, understandable pieces
- Your chats are saved, so you can revisit them anytime

If you're starting a new topic, tell me what you're working on, and I'll help you outright in.

5. Quiz Me! The all in one Practice Tool

If you want to study more actively, use the **Quiz Me** feature. You'll enter:

- Your name
- Your subject
- A list of topics you want to practice

I'll generate a **10-question quiz**, ask for your answers one at a time, and later let you know which ones to review. If you'd like, I'll explain the correct answers to the questions you missed. When you finish a topic, I can quiz you on the next one.

6. Managing Tasks and Calendar Events

To keep your semester organized, visit the **Tasks** or **Calendar** sections:

- Add homework, exams, or projects
- Track due dates
- View everything in one place
- Create calendar events for study sessions or classes

Everything you add stays synced with your account and securely stored.

7. Setting Up Your Class Schedule

In the **Schedule** section, you can enter:

- Course names
- Class times
- Days of the week
- Color coding for each course

Once your schedule is saved, I can help you plan study sessions around your actual availability.

You've Got This!

Your steps into becoming a more organized student will shine through your performance in classes! Have fun with the StudyBuddy app!



10.2. Appendix B- Screenshots

Hey John, wishing you a super day of learning – your brain is clearly "booked" for brilliance!

Here's what's coming up:

- Task: test on Dec 10, 12:00 PM

Today's Classes:

- Computer Science 1: 9:00 AM - 10:30 AM
- Calculus 2: 1:00 PM - 2:30 PM



Start a Study Session

Study Scheduler

Assignments & Tasks

Calendar

Practice / Quiz

My Calendar

← December 2025 →

Mon	Tue	Wed	Thu	Fri	Sat	Sun
	1 ●	2 ● ●	3 ●	4 ●	5 ●	6
7	8 ● STUD...	9 ● ●	10 ● REPO...	11 ● study... HOM...	12 ● quiz ...	13
14	15 ●	16 ● ●	17 ●	18 ●	19 ●	20
21	22 ●	23 ● ●	24 ●	25 ●	26 ●	27
28	29 ●	30 ● ●	31 ●			

December 11, 2025

Events & Tasks

- study 4 math 12:00 AM
- HOMEWORK Due at 12:00 AM HW

Today's Classes

- Biology 9:00 AM - 10:30 AM

You're doing fantastic! Using a timer is an incredibly effective strategy for focused learning and productivity – you're building great study habits right now. Keep up that brilliant work; your dedication will absolutely pay off!



Study Session

00:00:56

Pause

Quit

You're doing great! That timer is a fantastic strategy for focused work and productivity. Keep up that awesome discipline – every concentrated minute is building your understanding and bringing you closer to crushing your goals!



Short Break

00:00:54

Pause

Quit

John: My name is John and I need some help with my Biology homework.

 Hi John! I'm El, your study buddy. Biology is a fascinating subject, and I'm happy to help you work through your homework.

What topic are you currently studying in Biology, or what's the first question you'd like to tackle? No worries if it seems tricky, we'll break it down together! 😊

John: Could you explain cell mitosis for me? I'm confused about the anaphase phase

 That's a great question, John! Mitosis can definitely have a few tricky parts, and anaphase is one of those crucial stages.

Before we dive specifically into anaphase, could you tell me a little bit about what you already know or remember about the purpose of mitosis in general? What's the big picture goal of a cell going through this process? Thinking about that might help us set the stage for understanding anaphase's role! 😊

John: I remember mitosis being the cell splitting into two

 You've got the core idea perfectly, John! Mitosis is indeed all about one parent cell dividing to create two identical daughter cells. That's a fantastic starting point.

Now, to make sure those two new cells are identical, what has to happen to the cell's genetic material (its DNA) before it splits? Think about what each new cell will need. 😊



Profile

First Name: John

Last Name: Doe

Email: test@gmail.com

My Classes

-  **Computer Science 1**
MON, TUE at 9:00 AM - 10:30 AM
-  **Calculus 2**
TUE, WED at 1:00 PM - 2:30 PM
-  **Biology**
THU, FRI at 9:00 AM - 10:30 AM

Logout

10.3. Appendix C- Gantt Chart from our Project Plan

					Milestone #1				Milestone #2				Milestone #3				C-Day	
Phase	Tasks	Complete%	Current Status Memo	Assigned To	09/02	09/09	09/16	09/23	09/30	10/07	10/14	10/21	10/28	11/04	11/11	11/18	11/25	12/02
Requirements	Define requirements	0%		Kara, Ojerhomu	15													
	Allocate tasks	0%		All		10												
	Get sign off on requirements	0%		All			5	10										
Project design	Define tech required	0%		Jasper, Ojerhomu				10	5									
	AI research	0%		Kara			5	10	10									
	Front End design	0%		Kara				5	10	10								
	Back End design	0%		Jasper			5	5	10	10	10							
	Develop working prototype	0%		Jasper					10	15	15	5						
	Test prototype	0%		Jasper						5	10	5						
	Review prototype design	0%		All							10	5	10					
Development	Rework requirements	0%		Kara							5	10	10					
	Document updated design	0%		Kara, Ojerhomu								5	5	10	10			
	Test product	0%		Jasper									5	10	10			
Final report	Presentation preparation	0%		All												15	10	10
	Poster preparation	0%		All													10	
	Final report submission to D2L	0%		Ojerhomu														5
			Total work hours	365	15	10	5	30	25	40	40	50	30	30	20	35	10	25

10.4. Appendix D- Test Environment Map

